# Enhancing Software Quality Using Design Patterns

## HOTEL RESERVATION SYSTEM

## Object Oriented Analysis and Design

### CSA1104

Name: Dharshini
Reg.No: 192521408

# 1. System Problem Analysis

What is wrong with the current reservation design?

**DUPLICATED CODE**

Booking, cancellation, payment and notification logic can be repeated across classes.

**INEFFICIENT CREATION**

different parts of the application create objects separately, making the code harder to manage.

**TIGHT COUPLING**

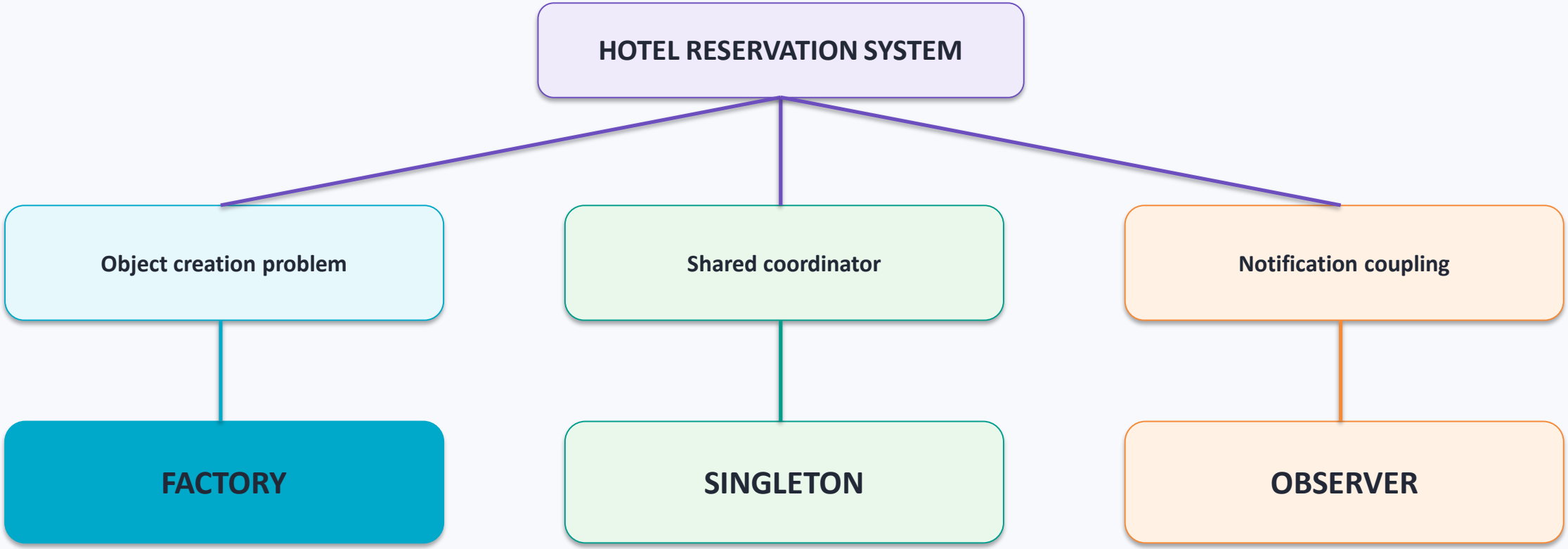
one class depends too much on another specific class. If we change one class, other parts may also need changes.




**Result: lower flexibility • higher maintenance effort • more change impact**

# 2. Pattern Selection

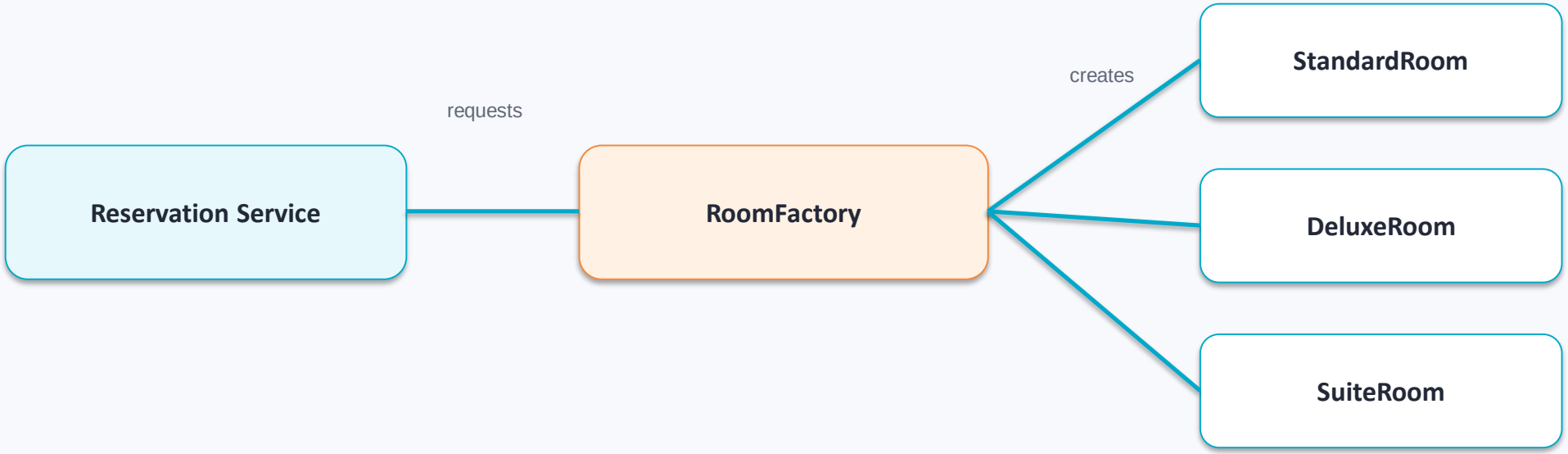
Match each recurring design problem with a suitable pattern




**Each pattern has a different responsibility — creation, controlled sharing, or event notification.**

# 3. Factory Pattern

Encapsulates object creation and hides concrete classes from the client




**BEFORE**

Client uses new StandardRoom(), new DeluxeRoom(), etc.

**AFTER**

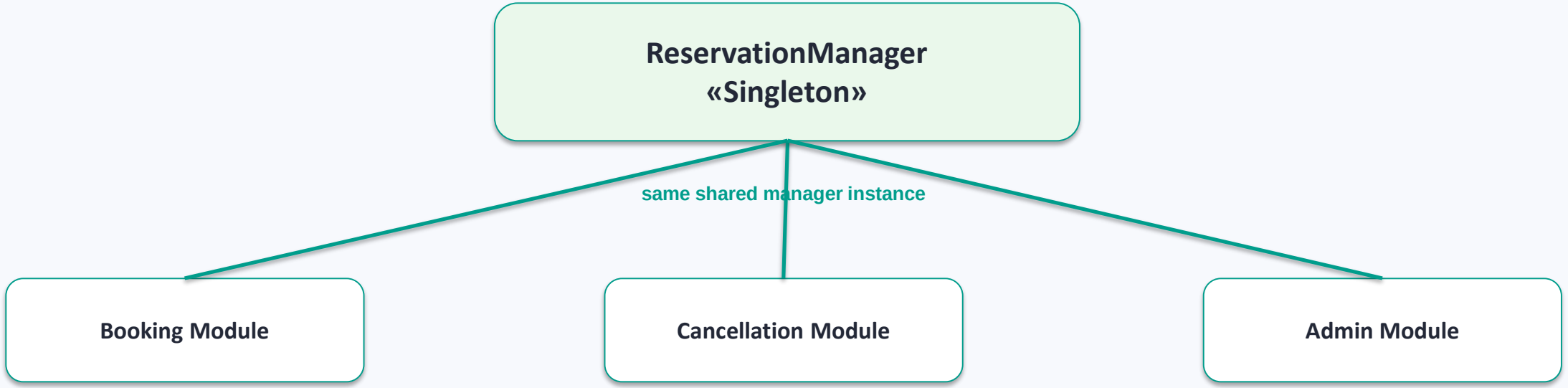
Factory decides which concrete room object to create.

**QUALITY GAIN**

Less coupling and easier addition of new room types.

# 4. Singleton Pattern

Provides one controlled instance for a genuinely shared coordinator




**WHY**

Avoids unnecessary duplicate manager objects.

**BENEFIT**

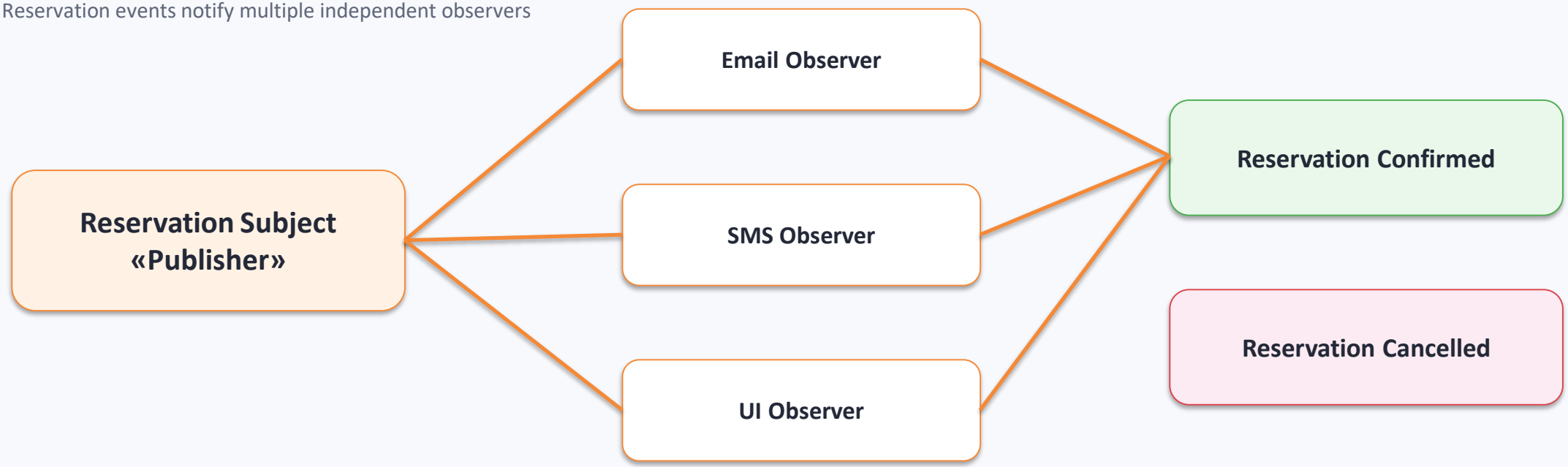
Common access point for shared coordination.

**CAUTION**

Do not overuse Singleton; it can create hidden dependencies.

# 5. Observer Pattern

Reservation events notify multiple independent observers




**One event → many reactions**

**EXTENSIBILITY**

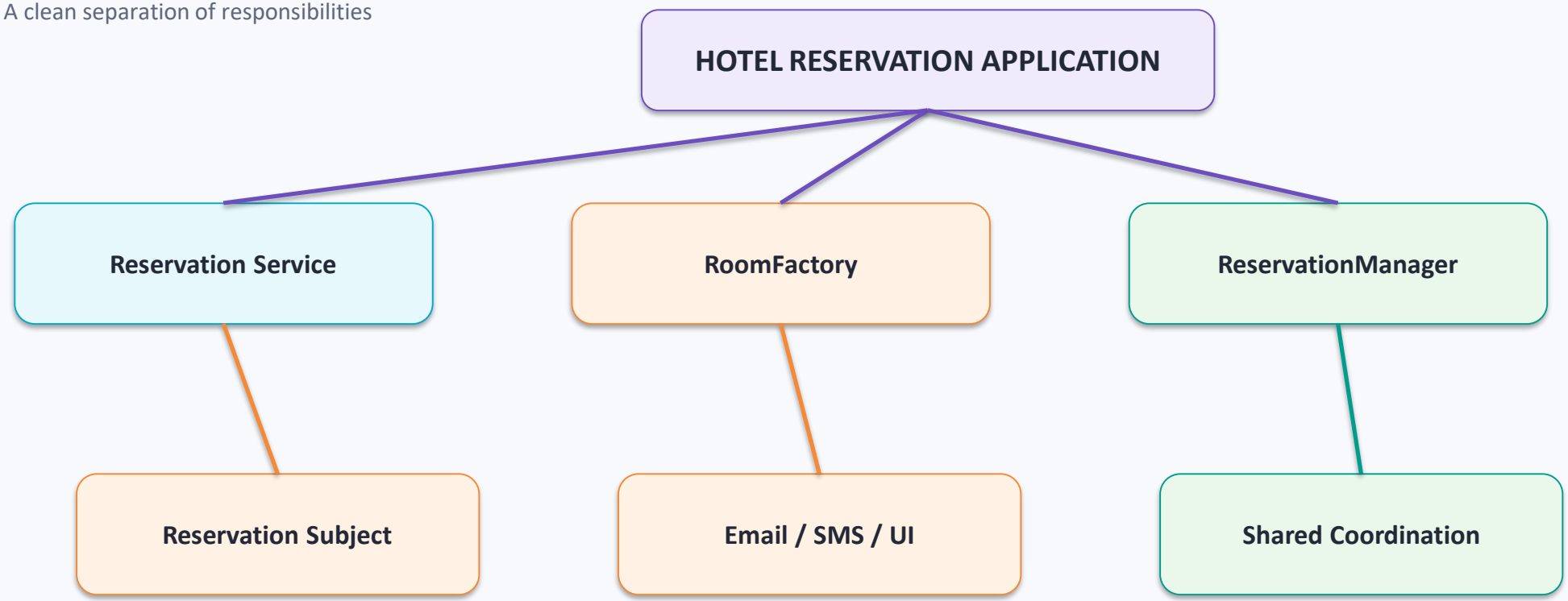
Add a new notification observer without changing reservation logic.

**LOW COUPLING**

Subject does not contain Email/SMS/UI implementation details.

# 6. Integrated Pattern Architecture

A clean separation of responsibilities




**Factory = creation | Singleton = controlled shared access | Observer = event notification**

# 7. Impact on Software Quality

How the patterns improve the requested quality attributes

### Flexibility

New room/notification types need fewer client changes.

### Maintainability

Changes are localized instead of duplicated.

### Reusability

Pattern structures can be reused for recurring problems.

### Testability

Clear responsibilities make components easier to isolate.

### Lower Coupling

Clients depend less on concrete implementations.

### Overall Quality

Better structure supports long-term software evolution.

# 8. Conclusion




## Design Patterns

**↓ Lower Coupling → Better Maintainability → Higher Software Quality**

The key is not using more patterns — it is using the right pattern for the right design problem.